

Horrible Code Activity

Bonus Git Hub Link:

<https://github.com/CharNyx8/redesigned-octo-bassoon.git>

About:

For this project we built a simple fortune teller site. It lets the user pick a category (Career, Love, Health, or Life), and then it gives them a random fortune from the selected category, three lucky numbers, and a lucky color.

Coding Principles:

1. KISS: the goal of this principle is to write code as simple as possible.

Function Names:

a. Bad:

- `def i_like_the_color_yellow_but_blue_is_so_pretty_yk(x):`
- `def why_was_six_scared_of_seven_cause_seven_eight_nine():`
- `def cool_beans_rando_stuff():`

b. Good:

- `def get_fortune`
- `def get_lucky_numbers`
- `def get_lucky_colors()`
- `def choose_fortune_category()`

In the bad version, the function names are poorly named, long, and unrelated to the job of the functions. This makes it hard to read and figure out what the code is actually supposed to do. On the other hand, the good version keeps it simple and straight to the point.

2. Single responsibility: this principle requires a class or module in a program to only have one specific functionality.
 - a. Bad:

```
def why_was_six_scared_of_seven_cause_seven_eight_nine():  # Kaleiah.M
    return sorted(random.sample(range(1, 51), k=5))
def cool_beans_rando_stuff():  # Kaleiah.M
    colors = ["Blue", "Green", "Orange", "Purple", "Gold"]
    return random.choice(colors)
def idk():  # usage  # Kaleiah.M
    whatever_xyz = input("Choose a category (career, love, health, life): ").lower()
    numbers = sorted(random.sample(range(1, 51), k=5))
    color = random.choice(["Blue", "Green", "Orange", "Purple", "Gold"])
    fortune = i_like_the_color_yellow_but_blue_is_so_pretty_yk(whatever_xyz)
    print(f"\nFortune: {fortune}")
    print(f"Lucky Color: {color}")
    print(f"Lucky Numbers: {numbers}")
if __name__ == "__main__":
    idk()
```

- Single responsibility is violated in the idk() function. This function is intended to get the user's input. Instead of just grabbing the user input, it grabs their fortune, lucky color, and lucky numbers

B. Good:

```
42 ✓ def get_fortune(category: str) -> str:
43     """
44     Return a random fortune string for the given category.
45
46     Args:
47         category: One of the keys in FORTUNES.
48
49     Returns:
50         A random fortune string for the given category or an error message if the category is unknown.
51     """
52     if category not in FORTUNES:
53         return f"Unknown category '{category}'. Choose from: {list(FORTUNES.keys())}"
54     return random.choice(FORTUNES[category])
55
```

- This is the correct version of the get fortune(). It takes the user input is separate and non-reliant on get a lucky number and get lucky color.

3. Clean Code: the goal of this principle is to have code that is focused, easy to read, and easy to maintain.

a. Bad:

- The bad version of the code has messy formatting, making it incredibly difficult to read and understand. It has poor variable names like whatever_xyz, RANDO, and idk. Also, it has inconsistent spacing, indentation, and overall really bad organization.

Ex:

```
def i_like_the_color_yellow_but_blue_is_so_pretty_yk(x):
    RANDO = {
        "career": ["You are getting fired.", "You are getting promoted!"],
        "love": ["Your crush will say Hi!", "You and your significant other need to break up.",
                "your significant other is going to propose."],
        "health": [
            "Your health is taking a turn for the worst.", "Your health is looking brighter!",
            "Lets get a bit healthier"],
        "life": [
            "Move to a new country!",
            "Everything will be better soon.",
            "A new adventure awaits. "]
    }
    if x not in RANDO:
        category = "life"
    return random.choice(RANDO[x])
def why_was_six_scared_of_seven_cause_seven_eight_nine():
    return sorted(random.sample(range(1, 51), 5))
def cool_beans_rando_stuff():
    colors = ["Blue", "Green", "Orange", "Purple", "Gold"]
    return random.choice(colors)
def idk():
    whatever_xyz = input("Choose a category (career, love, health, life): ").lower()
    numbers = sorted(random.sample(range(1, 51), 5))
    color = random.choice(["Blue", "Green", "Orange", "Purple", "Gold"])
    fortune = i_like_the_color_yellow_but_blue_is_so_pretty_yk(whatever_xyz)
    print(f"\nFortune: {fortune}")
    print(f"Lucky Color: {color}")
    print(f"Lucky Numbers: {numbers}")
if __name__ == "__main__":
    idk()
```

b. Good:

- Code is consistently formatted, easy to read, and built to be easy to maintain. Separation of concerns, single responsibility, and clear function names make it easy to maintain.

Ex:

```
def get_lucky_numbers(count: int = 3) -> List[int]:
    """
    Return a sorted list of 3 unique lucky numbers (1-99).

    Args:
        count: How many lucky numbers to generate (default 3).

    Returns:
        A sorted list of unique integers.
    """
    return sorted(random.sample(LUCKY_NUMBERS, count))

def get_lucky_colors() -> str:
    """
    Return a single random lucky color.
    """
    return random.choice(LUCKY_COLORS)

def choose_fortune_category():
    """
    Prompt the user to pick a valid fortune category from the available options.

    Keeps asking until a valid choice is entered.

    Returns:
        The chosen category string.
    """
    categories = list(FORTUNES.keys())
    print("\nAvailable fortune categories:")
    for i, category in enumerate(categories):
        print(f"{i+1}. {category}")
```

4. Document your Code (note common commenting errors): having comments in your code to explain objects, variables, functions etc.

a. Bad:

- There is no documentation provided at all. This makes it difficult for anyone who's trying to read and understand the code.

b. Good:

- Code is well-documented with a module docstring, function docstrings, and parameter / return descriptions.

```
def get_fortune(category: str) -> str:
    """
    Return a random fortune string for the given category.

    Args:
        category: One of the keys in FORTUNES.

    Returns:
        A random fortune string for the given category or an error message if the category is unknown.
    """
```

```
def get_lucky_colors() -> str:
    """
    Return a single random lucky color.
    """
    return random.choice(LUCKY_COLORS)

def choose_fortune_category():
    """
    Prompt the user to pick a valid fortune category from the available options.

    Keeps asking until a valid choice is entered.

    Returns:
        The chosen category string.
    """
```

5. Separation of Concerns: the goal of this principle is to have a program designed with different containers that don't have access to each other.

a. Bad:

- Data, core functions, and the user entry point are not neatly separated from each other.

```
def i_like_the_color_yellow_but_blue_is_so_pretty_yk(x):  
    RANDO = {  
        "career": ["You are getting fired.", "You are getting promoted!"],  
        "love": ["Your crush will say Hi!", "You and your significant other need to break up.",  
                 "your significant other is going to propose."],  
        "health": [  
            "Your health is taking a turn for the worst.", "Your health is looking brighter!",  
            "Lets get a bit healthier"],  
        "life": [  
            "Move to a new country!",  
            "Everything will be better soon.",  
            "A new adventure awaits. "]  
    }  
    if x not in RANDO:  
        category = "life"  
    return random.choice(RANDO[x])
```

b. Good:

- While there aren't different containers that don't have access to each other, the sections of code are separated from each other. The sections are data, core functions, and the user entry point to the program.

```
# Data  
  
FORTUNES = {  
    "career": ["A promotion is on the horizon.",  
               "Your hard work will be recognized soon.",
```

```
# Core Functions  
  
def get_fortune(category: str) -> str:
```

```
# Entry point  
  
def main() -> None:  
    """  
    Run the interactive fortune-teller session.  
    """
```